

Software configuration audits, part 2

Scripts, results and findings — Lab 3 Part B (**MC**-03 to **MC**-07)

420-D04-AB Conformity Assessment · Day 8 · Saturday, November 14, 2026 · 08:30–13:30 **EST**

AEC Cybersecurity (LEA.D8) · cohort CYB-01 · Fall 2026

Today's plan

- 08:30 Recall of Day 7
- 08:45 Lecture — six lessons
- 09:30 Break
- 09:45 Lab 3 Part B — guided start (B1–B3)
- 11:30 Micro-break
- 11:40 Lab 3 Part B — individual work (B4–B6) and questions

Day 8

KEY MESSAGE

Six lessons, then Lab 3 Part B until 13:30.

Rules for today

- Targets: your own **LAB-WIN11** and **LAB-UBUNTU** only — never the college, home or a classmate's **VM** (virtual machine)
- Inside your authorization period: Nov 11, 18:30 → Nov 19, 23:59 **EST** (Eastern Standard Time) (Lab 2 letter)
- Both kit scripts only **read** settings — allowed during Érable Cycles' business hours
- The Nmap reachability test waits for Tuesday, after 18:00
- No real personal information in any evidence file
- Legal Notice v2.0: §6.1 authorization · §6.4 lab rules · §7.4 changes · §7.5 findings

Day 8

KEY MESSAGE

Own VMs, authorization period, read-only.

Recall of Day 7 (1/2)

- **Configuration audit**: actual settings compared with an approved **baseline**
- Two runs over time reveal **configuration drift**
- **CIS-CAT** (Center for Internet Security Configuration Assessment Tool) Lite → **conformity score** (conform ÷ tested × 100)
- Lynis → **hardening index** (0–100)
- A score summarizes; the **pass rule** decides
- **CIS** (Center for Internet Security) Level 1 = essential · Level 2 = defence in depth · stricter value wins

Day 8

KEY MESSAGE

A score summarizes; the pass rule decides.

Recall of Day 7 (2/2)

- **MC-01 (Master Checklist)** score ≥ 80 % and no failed Level 1 item (account policy, firewall, audit)
- **MC-02** hardening index ≥ 70 and zero warnings
- **MC-08** no failed critical/high check on WEB-APP01 or DB-RDS01
- **MC-03** supported release · newest update ≤ 30 days · automatic updates on
- Unsupported **OS** (operating system) = non-conformity
- A FAIL on a fresh **VM** is a normal result → today it becomes a finding

Day 8

KEY MESSAGE

Wednesday's FAIL results are today's findings.

By the end of this hour you will be able to

- Read a check as check → expected → observed → verdict
- Run a PowerShell or Bash check script safely
- Interpret results: verdicts, false positives and negatives, compensating controls
- Write a finding with the columns of the findings table and rate it
- Tell a non-conformity, a deviation and an approved exception apart
- Close the loop: fix under change control, re-run, compare

Day 8

KEY MESSAGE

Six goals, one per lesson.

Lesson 1 — Scripts that automate configuration checks

Why a script · the four-part pattern · inside a PowerShell script · running it safely

Windows and PowerShell — about 9 minutes



Why a configuration check is automated

- 45 laptops × 10 settings ≈ 450 screens to read by hand
- **Repeatable** — anyone can re-perform the test
- **Consistent** — the same rule for every host
- **Evidence** — observed value, rule and verdict saved in a file
- The script collects and compares; the auditor **confirms** and **interprets**
- A wrong rule gives wrong verdicts — fast

PS>

KEY MESSAGE

Repeatable, consistent, evidence.

The four-part pattern of every check

- **Check** — what is tested, and where it is read
- **Expected value** — the **pass rule** from the baseline (≥, ≤, between, equals)
- **Observed value** — what the system returns, copied as is
- **Verdict** — PASS · FAIL · N/A (reason) · NOT TESTED (reason)
- With AND, one failed condition = FAIL
- Example W04: expected 1–5 and ≥ 15 min · observed 10 and 10 → FAIL

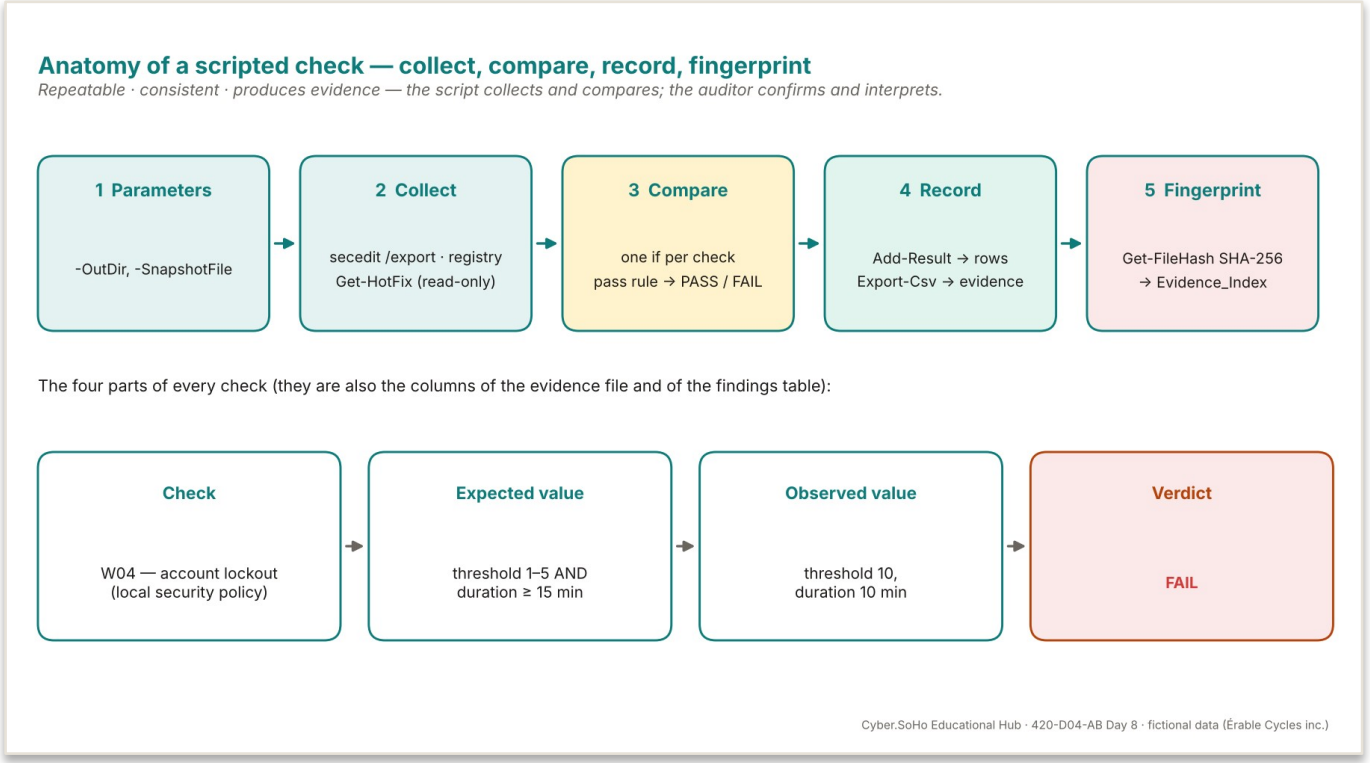
PS>

KEY MESSAGE

Check, expected, observed, verdict.

Inside a PowerShell check script

- **Cmdlets** named Verb-Noun: Get-HotFix, Get-FileHash
- **Variables** start with $ · the **pipeline** | passes objects
- 1 Parameters → 2 Collect (secedit, registry) → 3 Compare (one if per check)
- 4 Record (Add-Result, Export-Csv) → 5 Fingerprint (Get-FileHash)
- Collection separated from comparison: one function reads, the checks decide



Key message: Five parts: parameters, collect, compare, record, fingerprint.

Running a downloaded script safely

- 1 **Verify the fingerprint**: Get-FileHash = SHA256SUMS.txt, otherwise do not run
- 2 **Execution policy for this window only**: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
- 3 **Unblock-File** removes the downloaded-from-the-Internet mark
- 4 **Run as administrator** — reading the security policy needs it
- Execution policy = safety feature, not a security boundary
- Read-only: changes need approval, a test and a way back (§7.4)

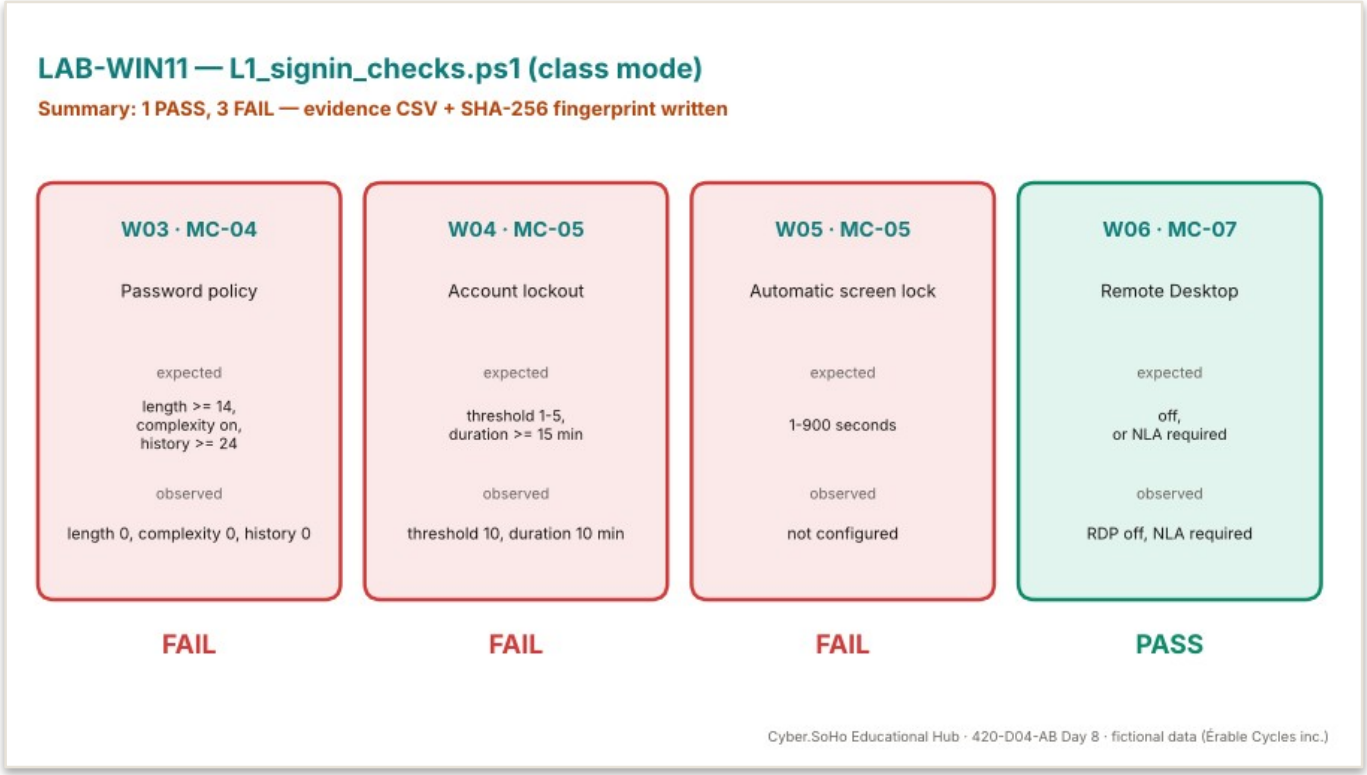
PS>

KEY MESSAGE

Fingerprint first; policy for this window only.

Example — four sign-in settings on LAB-WIN11

- Karim Haddad: “screen locks after 15 minutes, passwords are strong” — a statement is not evidence
- One read-only script reads W03–W06 (snapshot in class, live in the lab)
- W03 FAIL · W04 FAIL · W05 FAIL · W06 PASS → 1 PASS, 3 FAIL
- Deliverable: L1_LAB-WIN11_signin-checks.csv + **SHA-256** (Secure Hash Algorithm, 256-bit) → findings table (B5)



Key message: The script found three gaps the statement hid.

Code — L1_signin_checks.ps1 (W04 excerpt)

Read the two values, apply the rule, record one row

```
 73  # W04 (MC-05): read how many failed sign-ins lock the account (0 = the account never locks).
 74  $thr = [int](Read-Setting "LockoutBadCount")
 75  # Read how many minutes a locked account stays locked.
 76  $dur = [int](Read-Setting "LockoutDuration")
 77  # Apply the MC-05 lockout rule: threshold between 1 and 5 AND duration of 15 minutes or more.
 78  $r = if ($thr -ge 1 -and $thr -le 5 -and $dur -ge 15) { "PASS" } else { "FAIL" }
 79  # Record W04.
 80  Add-Result "W04" "MC-05" "Account lockout" "threshold 1-5, duration >= 15 min" "threshold
$thr, duration $dur min" $r
```

File: L1_signin_checks.ps1 — full listing, step-by-step runner and download in the lecture file.

Lesson 2 — Scripts that automate configuration checks on Linux

Bash for auditors · effective values · patch status on Ubuntu · ports open to the network

Linux and Bash — about 9 minutes



Bash for auditors — five pieces

- **Variable**: HOST="LAB-UBUNTU" (no spaces around =)
- **Command substitution**: state=$(systemctl is-enabled unattended-upgrades)
- **Pipe**: get ports | awk '{print $5}'
- **Test**: if ["$state" = "enabled"]; then r=PASS; else r=FAIL; fi
- **Safety options**: set -euo pipefail
- Bash passes **text**, PowerShell passes **objects**



KEY MESSAGE

Variables, $( ), pipes, tests, set -euo pipefail.

Read the effective value

- **Effective value** = what the program really uses after all its files are combined
- Ubuntu: sshd_config includes **sshd_config.d/*.conf** first
- For each keyword, sshd keeps the **first value** it reads
- sudo sshd -T prints the effective settings (lower case)
- 24.04 socket activation: sshd -T may fail → the kit reads the files
- without-password = old name of prohibit-password → still FAIL for “no”

$_

KEY MESSAGE

Drop-in files can override the main file.

Patch status on Ubuntu (MC-03)

- **L01** release still receives security updates (24.04 **LTS** (Long-Term Support): yes)
- **L02** no security update waiting (apt list --upgradable, -security lines)
- **L03** automatic security updates on (unattended-upgrades enabled — P6 §3)
- Old **package lists** → the count of waiting updates can be too low
- Step B2 starts with sudo apt update

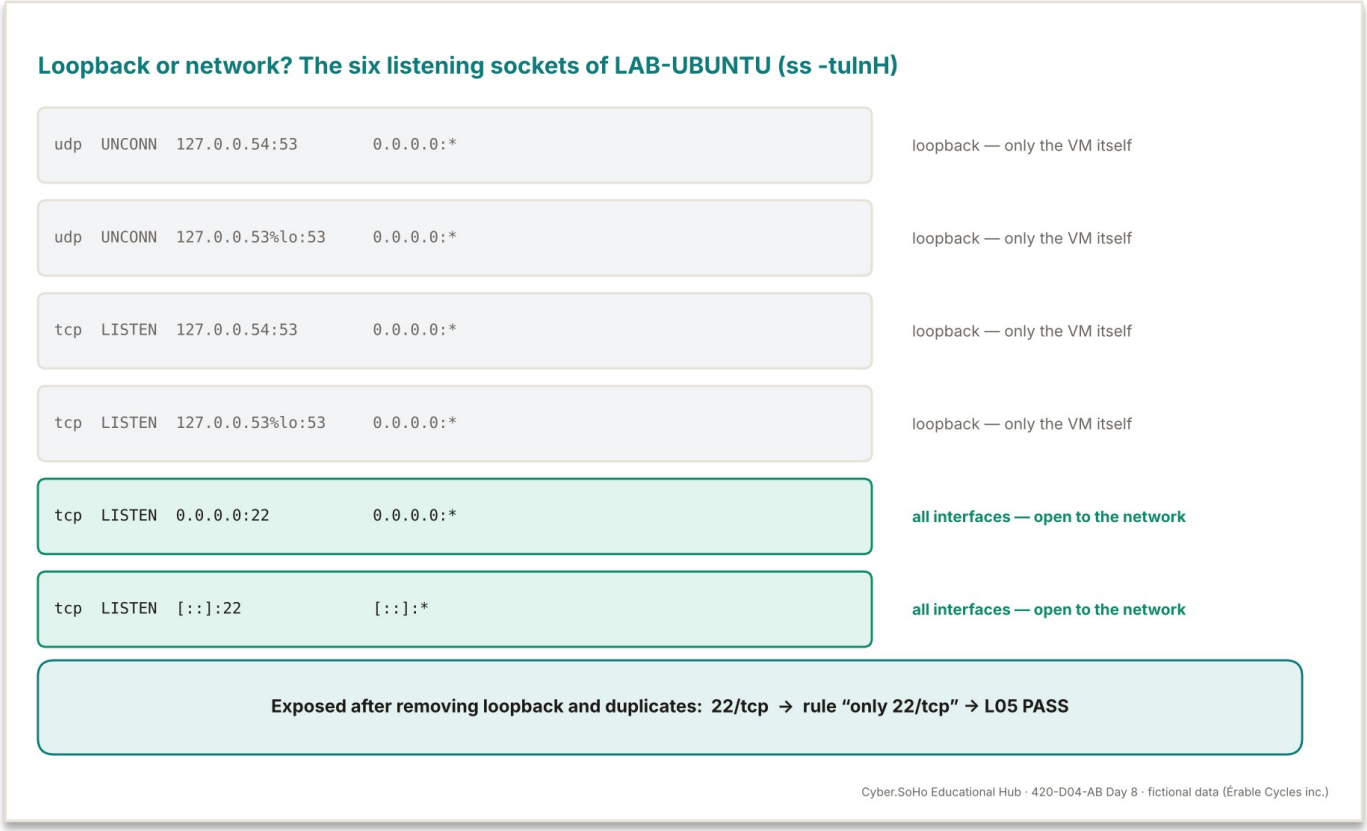


KEY MESSAGE

Refresh the package lists before counting.

Ports open to the network (MC-06)

- **Listening socket** = a program waiting on a port
- ss -tulnH: **TCP** (Transmission Control Protocol), **UDP** (User Datagram Protocol), listening, numeric, no header
- **127.x.x.x** and **[::1]** = loopback → only the **VM** (virtual machine) itself
- 0.0.0.0:22 and [::]:22 → every interface → open to the network
- 127.0.0.53 / .54 port 53 = local name resolution (ignored)
- Rule **MC**-06 (Master Checklist): only 22/tcp exposed



Key message: The address decides who can reach a port.

Example — is the web server really “locked down”?

- **MSP** (managed service provider): “**SSH** (Secure Shell) keys only, updates automatic”
- Five read-only checks L03–L07 on LAB-UBUNTU (stands in for WEB-APP01)
- L03 FAIL · L04 FAIL · L05 PASS · L06 FAIL · L07 PASS → 2 PASS, 3 FAIL
- Half of the statement was true
- Deliverable: L2_LAB-UBUNTU_checks.csv + **SHA-256** (Secure Hash Algorithm, 256-bit) → findings table

Where each Linux check reads its value (LAB-UBUNTU)

Check	Command or file (effective value)	Field kept	Pass rule	Observed	Verdict
L03 · MC-03	systemctl is-enabled unattended-upgrades	the whole output	= enabled	disabled	FAIL
L04 · MC-04	grep PASS_MAX_DAYS /etc/login.defs	column 2 (awk)	≤ 365	99999	FAIL
L05 · MC-06	ss -tulnH	non-loopback port/proto	= 22/tcp	22/tcp	PASS
L06 · MC-07	sudo sshd -T	permitrootlogin	= no	without-password	FAIL
L07 · MC-07	sudo sshd -T	passwordauthentication	= no	no	PASS

Class mode: get NAME prints saved/NAME.txt · Lab mode: get NAME runs the command above (sudo for sshd -T).

Cyber.SoHo Educational Hub · 420-D04-AB Day 8 · fictional data (Érable Cycles inc.)

Key message: Half of the statement was true.

Code — L2_ubuntu_checks.sh (L05 excerpt)

One pipe: drop loopback, keep port/protocol, remove duplicates

```
 72  # L05 (MC-06): keep the sockets that are not on a local-only (loopback) address, written as
port/protocol.
 73  exposed=$(get ports | awk '$5 !~ /^127\./ && $5 !~ /^\[::1\]/ { n = split($5, a, ":"); print
a[n] "/" $1 }' | sort …
 74  # PASS when the only port reachable from the network is 22/tcp (SSH, the approved
administration port).
 75  if [ "$exposed" = "22/tcp" ]; then r=PASS; else r=FAIL; fi
 76  # Record L05.
 77  record L05 MC-06 "Ports open to the network" "only 22/tcp" "$exposed" "$r"
```

File: L2_ubuntu_checks.sh — full listing, step-by-step runner and download in the lecture file.

Lesson 3 — Reading and interpreting the results

Verdicts and reasons · false positives and false negatives · corroboration · compensating controls · high risk first

About 9 minutes



Four verdicts, and when a reason is mandatory

- **PASS** — rule met (the observed value is the proof)
- **FAIL** — rule not met (confirm it)
- **N/A** — requirement does not apply → reason tied to the scope
- **NOT TESTED** — test could not run → reason + how it will be completed
- No reason = treated as missing work
- An honest “Not tested — reason …” costs nothing; an invented result fails the item

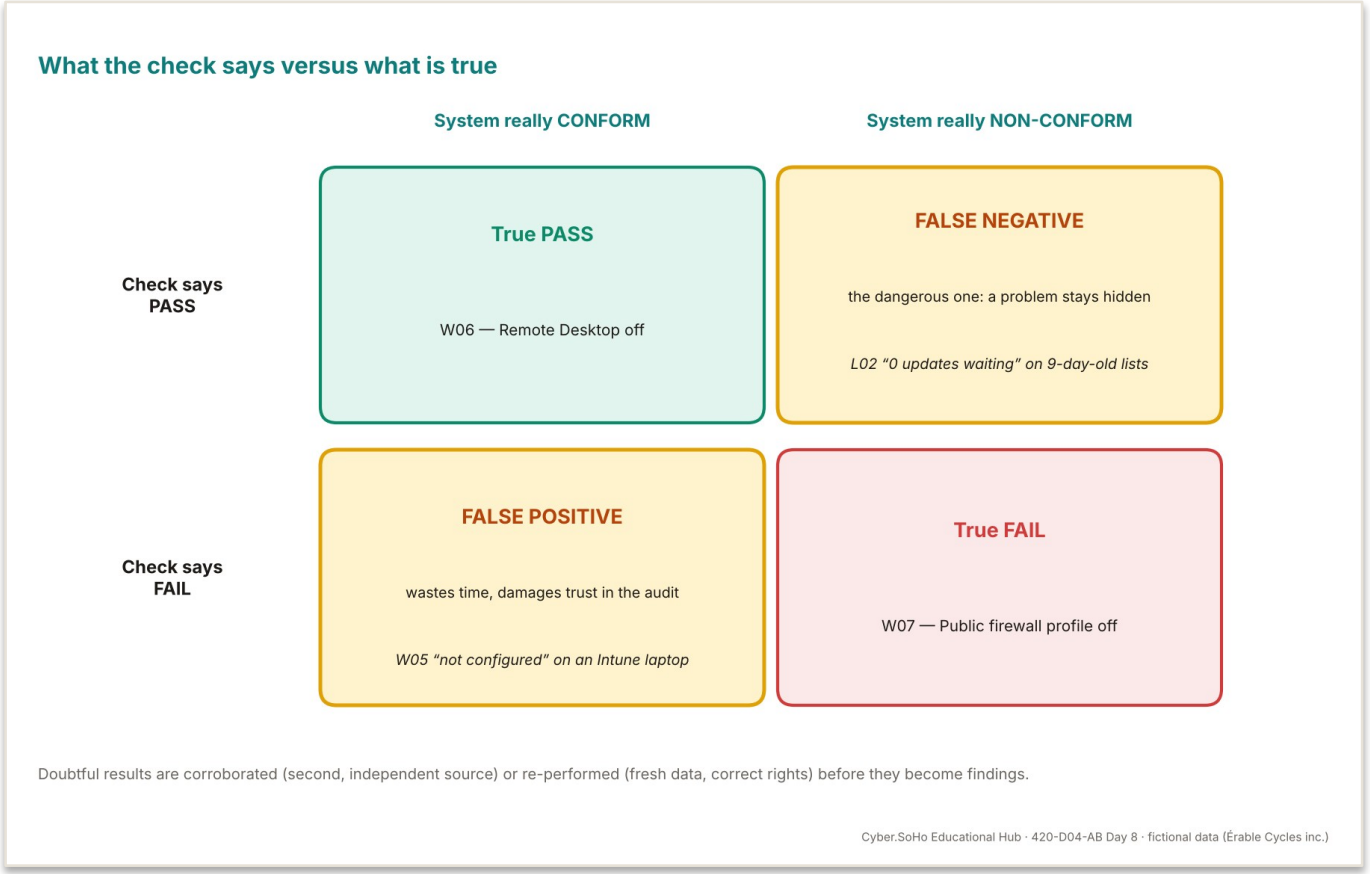


KEY MESSAGE

N/A and NOT TESTED always carry a reason.

False positives and false negatives

- **False positive** = FAIL while the system is conform → wastes time, damages trust
- **False negative** = PASS while the system is non-conform → the dangerous one
- Causes: wrong source, old data, different format, missing rights
- W05 “not configured” on an Intune laptop → possible false positive
- L02 “0 waiting” on 9-day-old lists → possible false negative



Key message: A false negative hides a real problem.

Corroboration — a second, independent source

- Doubtful result → confirm with a source that does not depend on the first
- W05: Intune device-policy report, or a timed **observation** (idle 16 minutes)
- L02: sudo apt update, then **re-perform** the check
- W03/W04 NOT TESTED (no admin rights) → re-run as administrator
- Write what you did in the note column — it becomes evidence



KEY MESSAGE

Re-perform or corroborate before recording.

Compensating controls

- A different measure that reduces the **same** risk
- L06: root may sign in with a key, but **SSH** (Secure Shell) is reachable only from the admin **VPN** (virtual private network)
- Verdict stays **Non-conform** — the requirement is still not met
- The control is written in the note and may lower the likelihood
- **PCI DSS** (Payment Card Industry Data Security Standard) Appendix B: meets the intent, similar defence, beyond other requirements

✓ ✗ ?

KEY MESSAGE

A FAIL with a compensating control is still a FAIL.

High risk first

- Confirmed failures are handled by **risk**, not by print order
- Likelihood (1–3) × impact (1–3) → 1–2 Low · 3–4 Medium · 6 High · 9 Critical
- Today's work order: W02 · W07 · W09 · L03 · L08 (6) → W10 · L06 (4) → L04 (2)
- Vulnerability management (next course): **CVSS** (Common Vulnerability Scoring System) and the CISA **KEV** (Known Exploited Vulnerabilities) catalog
- Step B4 = this triage, before the findings table

✓ ✗ ?

KEY MESSAGE

Triage first, findings second.

Example — twenty results, five lanes

- First run: W03/W04 NOT TESTED (no admin) · L02 PASS on old lists · W05 asks for a second source · L09 N/A without reason
- Lanes: Re-run 3 · Corroborate 1 · Fix the record 1 · Confirmed FAIL 8 · Conform 7
- Ranked: W02, W07, W09, L03, L08 (6) › W10, L06 (4) › L04 (2)
- Deliverable: L3_triage.csv — the work order of step B4



Key message: Four rows had to be fixed before any finding was written.

Code — L3_interpret_results.py (rules excerpt)

Seven rules in order; the first that matches decides

```
    # Rule 1 - NOT TESTED because the script lacked rights: the test is valid only after a correct re-run.
    if result == "NOT TESTED" and "administrator" in note:
        # Lane and action for this case.
        return "Re-run first", "run the script as administrator, then replace this row"
    # Look for "refreshed N day(s) ago" in the note; m is None when the note does not say it.
    m = re.search(r"refreshed (\d+) day", note)
    # Rule 2 - a PASS computed from data older than one day may hide waiting updates (possible false negative).
    if result == "PASS" and m and int(m.group(1)) > 1:
        # Lane and action for this case.
        return "Re-run first", "possible false negative: run 'sudo apt update', run the script again"
    # Rule 3 - the note itself says the value may come from another mechanism (possible false positive).
    if result == "FAIL" and "second source" in note:
        # Lane and action for this case.
        return "Corroborate", "possible false positive: confirm with an independent second source"
```

File: L3_interpret_results.py — full listing, step-by-step runner and download in the lecture file.

Lesson 4 — Documenting configuration findings

What a finding is · one finding = one issue · the 5 Cs · the columns · severity · the Lab 3 rule

About 9 minutes



What a finding is

- **Audit finding** = evidence compared with the **audit criteria**
- It shows **conformity** or **non-conformity**
- Conform rows prove what works; non-conform rows show what to correct
- **One finding per issue** — its own evidence, owner, fix and closing date
- Lab 3 rule: ≥ 12 findings across both VMs, ≥ 3 Conform

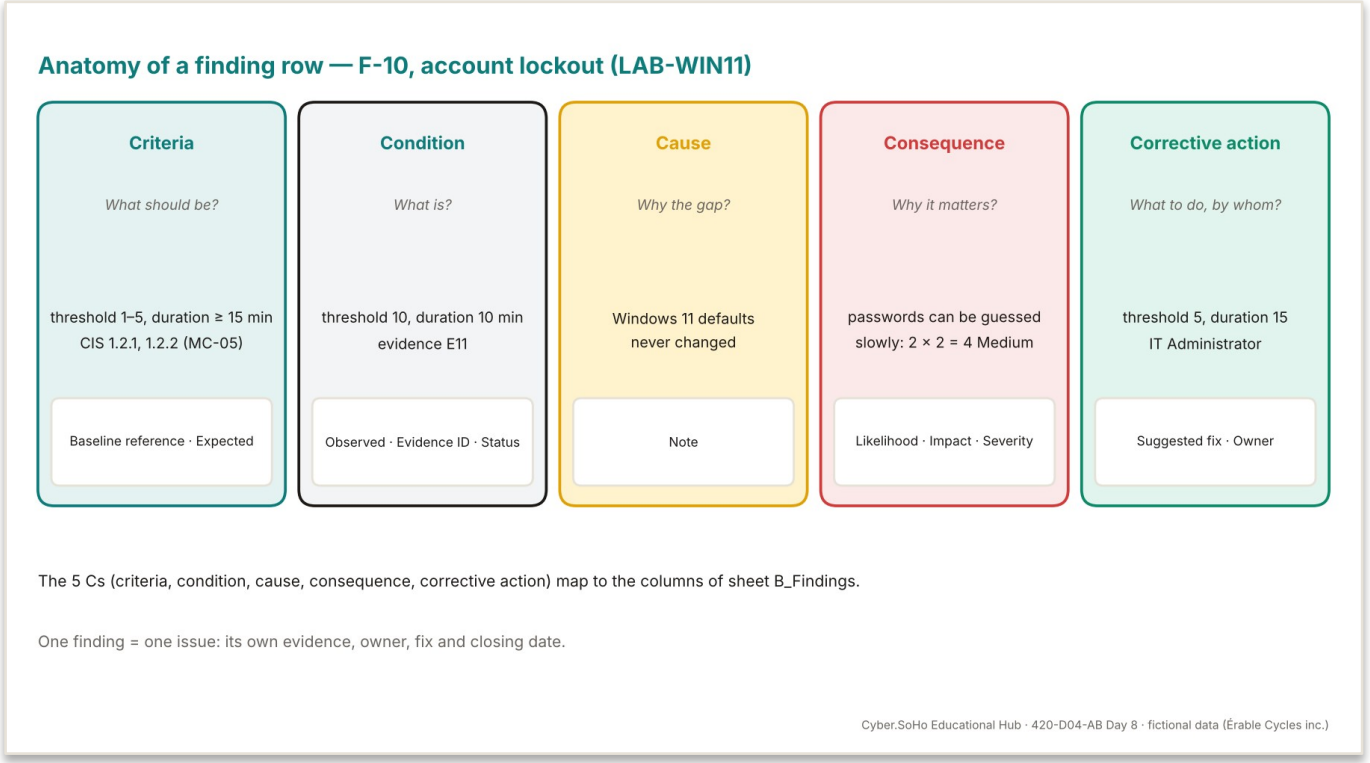
F-10

KEY MESSAGE

Findings include conform rows.

The five elements of a finding — the 5 Cs

- **Criteria** — what should be → baseline reference, expected
- **Condition** — what is → observed, evidence **ID** (identifier), status
- **Cause** — why the gap → note
- **Consequence** — why it matters → likelihood, impact, severity
- **Corrective action** — what to do, by whom → fix, owner



Key message: Criteria, condition, cause, consequence, corrective action.

The columns of the findings table

- Finding **ID** · **MC** (Master Checklist) item · Host · Check
- **Baseline reference** — **CIS** (Center for Internet Security) number, policy clause or **MC** rule
- Expected · Observed · **Evidence ID** (L3-Exx)
- Status — Conform / Non-conform / N/A
- Likelihood × Impact → Severity (Non-conform only)
- Suggested fix · Owner · Note
- A **draft** policy (P5 v0.9) is not a criterion — use **CIS**, mention the draft

F-10

KEY MESSAGE

Every column has a stated source.

Rating severity

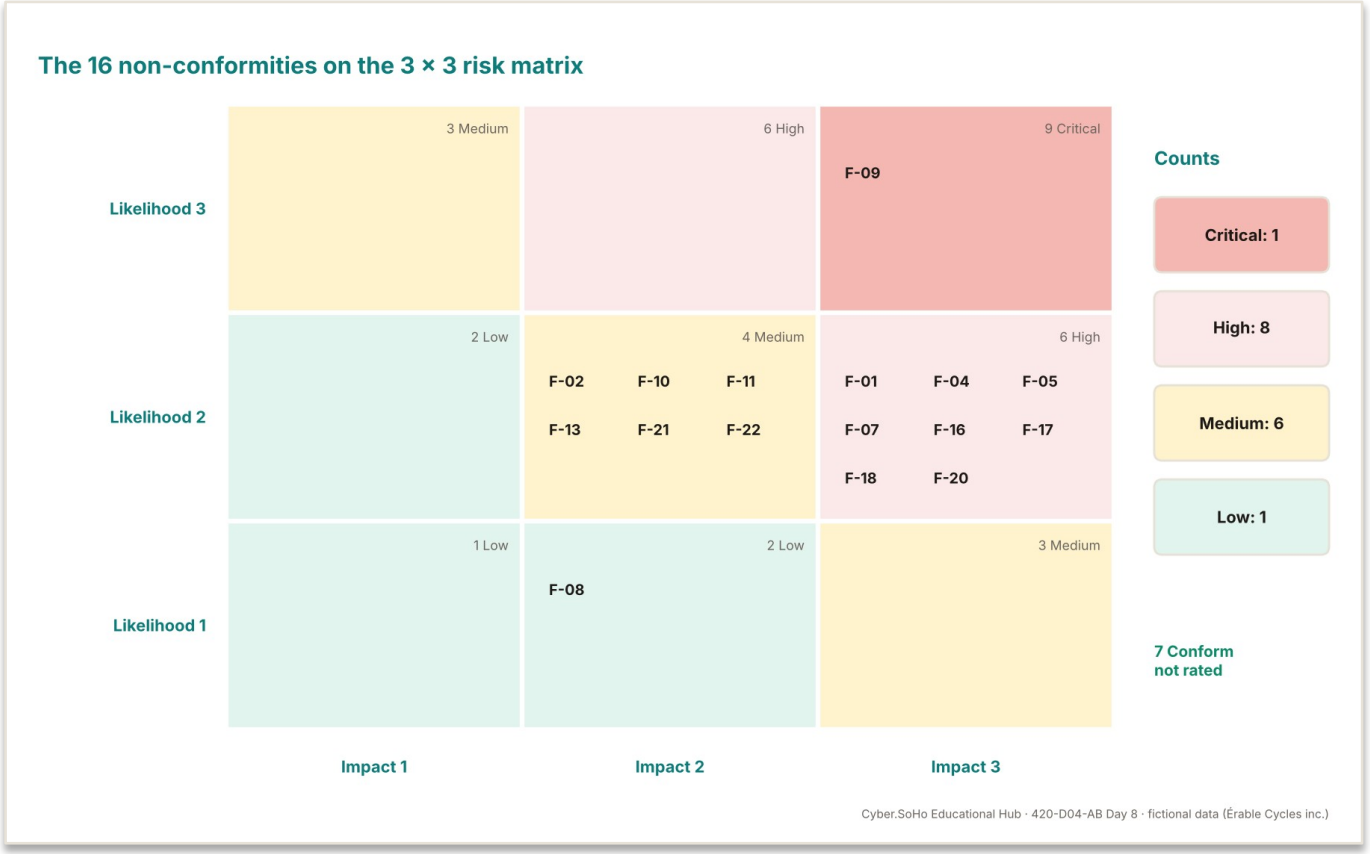
- Likelihood 1 unlikely · 2 possible · 3 likely
- Impact 1 low · 2 medium · 3 high
- Score = L × I → 1–2 Low · 3–4 Medium · 6 High · 9 Critical
- A compensating control can lower the likelihood
- A legal duty breached → impact 3
- Rate only Non-conform rows

	Impact 1	Impact 2	Impact 3
Likelihood 3	3 Medium	6 High	9 Critical
Likelihood 2	2 Low	4 Medium	6 High
Likelihood 1	1 Low	2 Low	3 Medium

Key message: Only non-conformities are rated.

Example — a findings table the CEO can read

- 23 results (20 script + 3 Part A) → F-01 … F-23
- 23 findings, 7 Conform → **rule met**
- 1 Critical (F-09 password policy) · 8 High · 6 Medium · 1 Low
- Conformity 7/23 = 30.4 %
- Deliverable: L4_findings_table.csv / .xlsx → sheet B_Findings → Project §6A



Key message: The password rules of the laptops come first.

Code — L4_findings_table.py (one finding)

Status, likelihood and impact only for Non-conform, then all the columns

```
 45      # The rule row that belongs to this check.
 46      rule = rules[r["CheckID"]]
 47      # The status word for this verdict.
 48      status = STATUS[r["Result"]]
 49      # Likelihood and impact exist only for non-conformities; a conform row is not rated.
 50      lik, imp = (int(rule["Likelihood"]), int(rule["Impact"])) if status == "Non-conform" else (None, None)
 51      # Score = likelihood x impact (empty for a conform row).
 52      score = lik * imp if lik else None
 53      # Assemble the row with the columns of sheet B_Findings.
 54      findings.append({"Finding ID": f"F-{number:02d}", "MC item": r["MC"], "Host": r["Host"], "Check":
r["Check"],
 55                       "Baseline reference": rule["Baseline"], "Expected": r["Expected"], "Observed":
r["Observed"],
 56                       "Evidence ID": rule["Evidence"], "Status": status, "Likelihood": lik or "", "Impact": imp
or "",
 57                       "Score": score or "", "Severity": severity(score) if score else "", "Suggested fix":
rule["Fix"],
 58                       "Owner": rule["Owner"], "Note": r["Note"]})
```

File: L4_findings_table.py — full listing, step-by-step runner and download in the lecture file.

Lesson 5 — Deviations and exceptions

Non-conformity · deviation · exception · risk acceptance · the eight elements · the register

About 9 minutes



Four words students mix up

- **Non-conformity** — a requirement not met (a fact)
- **Deviation** — a recorded gap, tracked in a register until closed
- **Exception** — formal, approved, time-limited permission not to meet a requirement, with compensating controls
- **Risk acceptance** — an informed decision by a person with authority
- An exception does **not** make the finding Conform

EX-01

KEY MESSAGE

The finding stays Non-conform, with the exception ID.

The eight elements of a valid exception

- 1 Finding and requirement · 2 Specific business reason · 3 Compensating control
- 4 Risk before/after — residual never Critical · 5 Owner
- 6 Approver with authority, **not the requester** · 7 End date ≤ 90 days · 8 Review date
- P6 §5 already asks for: reason, owner, end date
- **ISO/IEC** (International Organization for Standardization / International Electrotechnical Commission) 27001 6.1.3: risk owners accept residual risk

EX-01

KEY MESSAGE

All eight, or rejected.

Who approves, and what happens at the end date

- Approver = can accept risk for the organization → the **CEO** (Chief Executive Officer) (Mathilde Arsenault)
- The **IT** (information technology) Administrator may request or recommend — not approve his own
- End date reached → requirement met and **verified** (deviation closed)
- … or a **new request** with the eight elements
- An expired exception = an open non-conformity

EX-01

KEY MESSAGE

Nobody approves their own exception.

The deviation register

- Every request, **approved or rejected**
- **ID** (identifier) · finding · requirement · reason · compensating control
- Risk before/after · owner · approver · start · end · review · status
- Evidence for the auditor: the organization manages its gaps
- Follow-up tool: every open line has a date
- Project §8 and §9 reuse the end and review dates

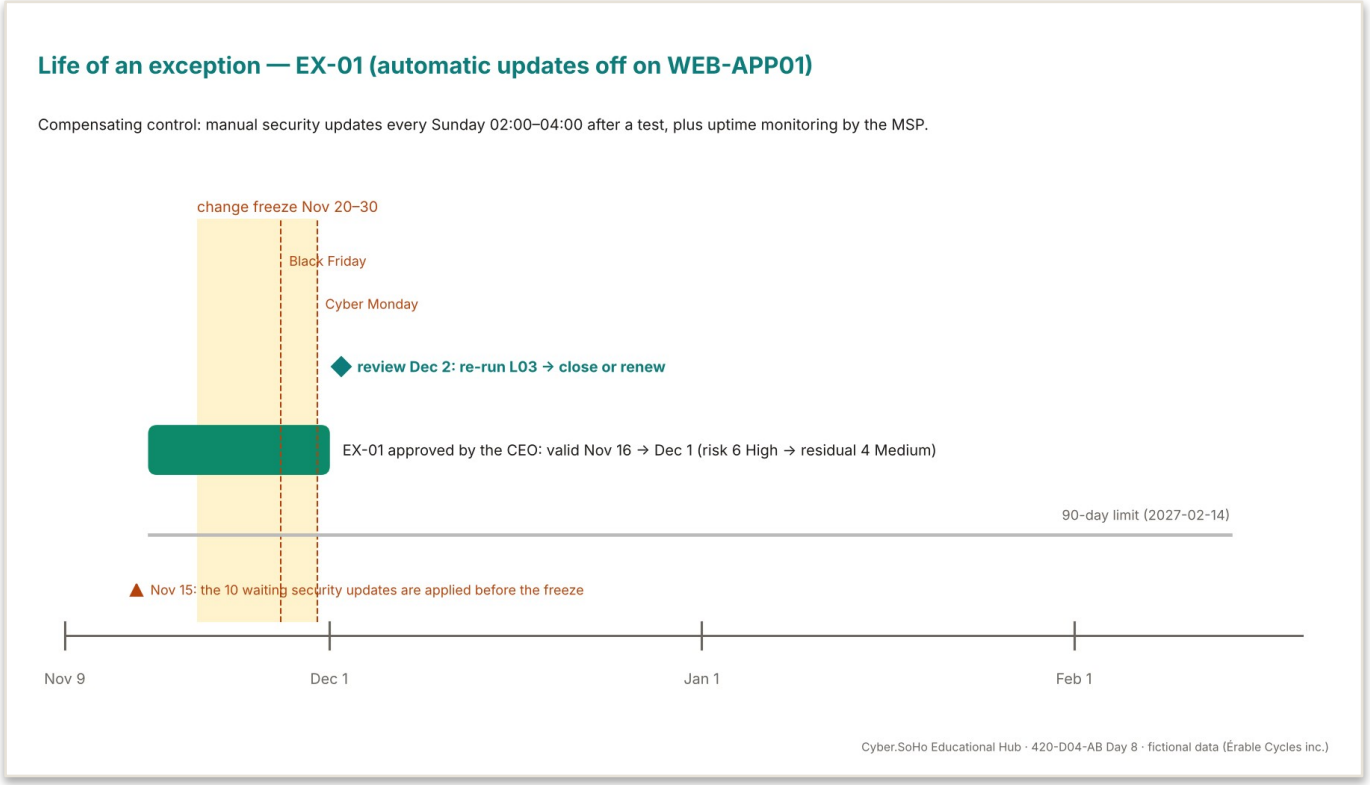
EX-01

KEY MESSAGE

Rejected requests stay in the register.

Example — Black Friday versus automatic updates

- EX-01: updates off on WEB-APP01 during the Nov 20–30 freeze · **MSP** (managed service provider) patches by hand on Sundays · risk 6 → 4
- Approved by the **CEO** · valid Nov 16 → Dec 1 · re-test Dec 2 → **APPROVED** (8/8)
- EX-02: keep weak passwords “because users complain” · no control · no end date · self-approved
- → **REJECTED** (2/8)



Key message: EX-01 is managed risk; EX-02 is a permanent hole.

Code — L5_exception_checker.html (the eight tests)

Each element is a small test function; the decision rule is all eight

```
117  const CRITERIA = [
118    // 1. The finding and the requirement it breaks must be named.
119    {label: "Finding and requirement identified", test: r => Boolean(r.finding && r.requirement)},
120    // 2. A specific business reason (convenience is not a reason).
121    {label: "Specific business reason", test: r => Boolean(r.reason) && !/complain|convenien|annoy/i.test(r.reason)},
122    // 3. A compensating control that reduces the same risk in another way.
123    {label: "Compensating control described", test: r => Boolean(r.compensating)},
124    // 4. Risk rated before and after; the residual risk must not be Critical (above the appetite of the CEO).
125    {label: "Risk rated, residual not Critical", test: r => r.riskBefore > 0 && r.riskAfter > 0 && r.riskAfter < 9},
126    // 5. An owner who is accountable for the risk while the exception lasts.
127    {label: "Owner named", test: r => Boolean(r.owner)},
128    // 6. Approved by a person with authority who is not the person who asked (independence).
129    {label: "Approver has authority and is not the requester", test: r => r.approverHasAuthority && r.approver !== r.…
130    // 7. An end date no later than 90 days after the start (Patch Management Procedure P6 s.5 asks for an end date).
131    {label: "End date set, at most 90 days", test: r => r.end !== null && days(r.start, r.end) <= 90},
132    // 8. A review date (re-test) no later than 7 days after the end date.
133    {label: "Review / re-test date set", test: r => r.review !== null && r.end !== null && days(r.end, r.review) <= 7}
134  ];
```

File: L5_exception_checker.html — full listing, step-by-step runner and download in the lecture file.

Lesson 6 — Closing the loop: Check → Act

PDCA in the audit · remediating under change control · re-run and compare · act for the whole organization

About 9 minutes



Where today's work sits in PDCA

- **Plan** — the assessment plan (Lab 2)
- **Do** — Érable Cycles runs its systems
- **Check** — scripts and findings table (today)
- **Act** — corrective actions of the organization
- **ISO/IEC** (International Organization for Standardization / International Electrotechnical Commission) 27001 10.2: correct, then **review the effectiveness**
- Real audit: the auditor recommends, the organization corrects (**independence**)



KEY MESSAGE

The audit is Check; the organization does Act.

Remediate safely — change control

- **Evidence first**: all evidence saved and fingerprinted; FAIL rows never overwritten
- **Snapshot** before-fix = the **backout plan**
- **Change ticket** naming the checks to fix (W04, L08) and the commands
- **Smallest correct change**
- net accounts /lockoutthreshold:5 /lockoutduration:15 /lockoutwindow:15
- sudo ufw allow 22/tcp && sudo ufw enable



KEY MESSAGE

Evidence, snapshot, ticket, smallest change.

Re-run, compare, verify

- Same script, same rules — then compare check by check
- **Fixed** FAIL → PASS · **Unchanged** · **Regressed** PASS → FAIL · **Unplanned change**
- Verified only if every planned check is fixed **and nothing else moved**
- Evidence E13: L3-E13_MC-yy_rerun_<host>.csv
- W03 needs three settings — /minpwlen:14 alone stays FAIL
- W04 is fixed by one command → the safe bonus choice



KEY MESSAGE

Verified = planned fixed, nothing else moved.

Act for the whole organization

- Fixing one **VM** (virtual machine) proves the fix — it does not fix 45 laptops
- Real corrective action: change the **baseline** (group policy on SRV-DC01 or Intune profile)
- Then re-test a **sample**
- Otherwise drift returns with the next new laptop
- Project §8 corrective actions · §9 follow-up re-test (e.g. 30 days later)

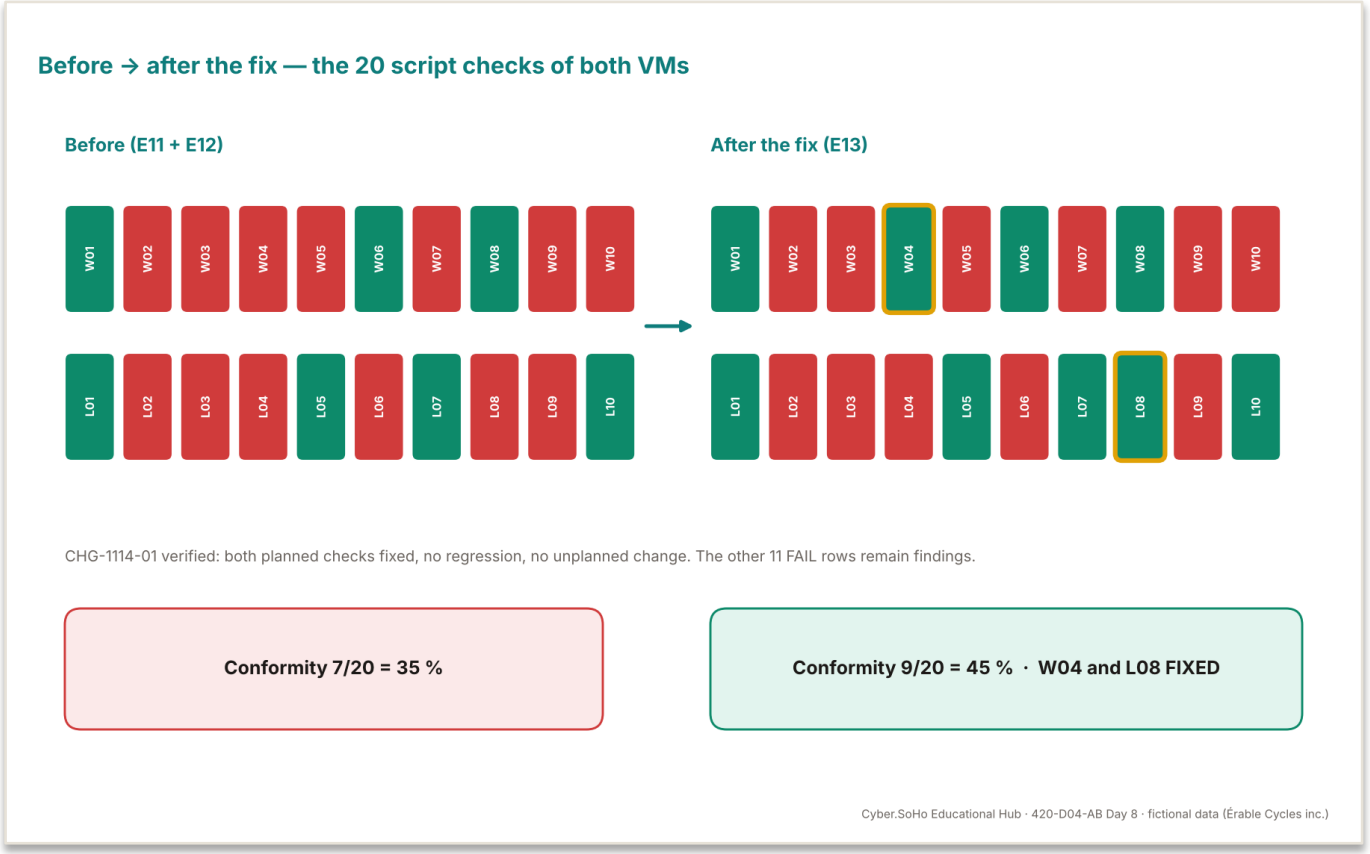


KEY MESSAGE

Act on the baseline, not on one machine.

Example — did the fix work?

- CHG-1114-01: snapshot before-fix, fix W04 (LAB-WIN11) and L08 (LAB-UBUNTU)
- W04 FAIL → PASS · L08 FAIL → PASS — both **verified**
- No regression, no unplanned change
- Conformity of the 20 script checks: 35 % → 45 %
- The other 11 FAIL rows remain findings · Deliverable: E13



Key message: Two planned fixes, nothing else moved: verified.

Code — L6_rerun_compare.py (classification)

FIXED, REGRESSED or UNCHANGED, then compared with the ticket

```
47      # Classify the change: FAIL -> PASS is fixed, PASS -> FAIL is a regression, same verdict is unchanged.
48      change = "FIXED" if (v1, v2) == ("FAIL", "PASS") else "REGRESSED" if (v1, v2) == ("PASS", "FAIL") else
"UNCHANGED"
49      # Compare the change with the ticket: planned and fixed = verified; planned and not fixed = not effective.
50      if cid in planned:
51          # Verdict on the planned change.
52          check = "verified" if change == "FIXED" else "fix not effective - investigate"
53      # A change that the ticket did not plan must be explained (drift or an undeclared change).
54      elif change != "UNCHANGED":
55          # Verdict on the unplanned change.
56          check = "UNPLANNED change - investigate"
57      # No change and not planned: nothing to explain.
58      else:
59          # Empty verdict.
60          check = ""
```

File: L6_rerun_compare.py — full listing, step-by-step runner and download in the lecture file.

Lab 3 Part B at a glance (1/2)

- **B1** copy and verify the kit (SHA256SUMS.txt)
- **B2** Linux checks L01–L10 → E10, E12 (sudo apt update first)
- **B3** Windows checks W01–W10 → E09, E11 (administrator)
- **B4** read the results — confirm, re-run, corroborate, justify
- **B5** sheet B_Findings — ≥ 12 findings, ≥ 3 Conform
- **B6** optional bonus (+0.5) — snapshot, fix two, re-run → E13

B1–B6

KEY MESSAGE

B1 to B6; guided start at 09:45.

Lab 3 Part B at a glance (2/2)

- Evidence names: L3-Exx_MC-yy_<what>_<host> + **SHA-256** (Secure Hash Algorithm, 256-bit) in Evidence_Index
- No real personal information in any file
- Keep all script files: W07–W10 and L08–L10 are Part C data (Tuesday)
- Grading A–B: 5.5 of 10 — tools 2 · scripts 1.5 · findings 1.5 · summary 0.5
- Lab 3 due Fri Nov 20, 23:59 **EST** (Eastern Standard Time) · testing ends Thu Nov 19, 23:59 **EST**

B1–B6

KEY MESSAGE

Keep every file; Part C reuses them.

Homework and announcements

- **Lab 2 due tomorrow** — Sunday, Nov 15, 23:59 **EST** (LÉA DropBox, .xlsx + .pdf)
- Paste today's procedure into **CPM** (Course Procedure Manual) **Chapter 4** (7-line template)
- Tuesday, Nov 17 (Day 9): Lab 3 Part C — verification of security controls
- Bring the case file: Exhibits D, E, F (zones, firewall rules, access review)
- Final Exam: about 10 of 40 questions on configuration audits

B1–B6

KEY MESSAGE

Lab 2 is due tomorrow night.

Videos for self-study (all 15 minutes or shorter)

- L1 Scripting and Automation (8:20) · Scripting Languages (4:47)
- L2 Bash in 100 Seconds (2:33) · Hardening Targets (10:11)
- L3 Analyzing Vulnerabilities (10:29) · Security Controls (11:48)
- L4 Writing an Audit Finding — **IIA** (Institute of Internal Auditors) (4:44) · Risk Management (3:30)
- L5 Risk Management Strategies (3:12) · Vulnerability Remediation (8:45)
- L6 Change Management (11:22) · **PDCA** (Plan-Do-Check-Act) Explained (3:53)

All twelve links (YouTube) are in the notes of this slide and in the lecture file; all confirmed online on November 9, 2026.

Key message: Twelve short videos, two per lesson.

Key links and legal

- Cyber.SoHo Legal Notice, Terms of Use and AUP v2.0
- **CIS** (Center for Internet Security) Benchmarks · **CIS** Controls v8.1 · **NIST** (National Institute of Standards and Technology) **SP** (Special Publication) 800-128
- **PCI DSS** (Payment Card Industry Data Security Standard) v4.0.1 (PCI SSC Document Library) · **ISO/IEC** (International Organization for Standardization / International Electrotechnical Commission) 27001
- Microsoft Learn: execution policies · Get-FileHash · Windows 11 release information
- Ubuntu automatic updates · sshd_config(5) · ss(8)
- Check every link with VirusTotal before you open it (§14, Annex C)

Legal Notice: https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html

Key message: Official sources only; check links with VirusTotal.



Questions — then break until 09:45

Recap: automate · interpret · document · accept formally · verify

Next: Lab 3 Part B, guided start

Tuesday: Part C — security controls